

Asistente multiagente para compra de componentes electrónicos

POC funcional con Streamlit, LangGraph, RAG local, fallback web, verificación LLM, observabilidad y evaluación reproducible.

Este informe sintetiza lo implementado en el repositorio y lo organiza de acuerdo con la consigna: objetivo, arquitectura, implementación técnica, evaluación de resultados, conclusiones y planificación del equipo.

Integrantes:

- Kevin Cajachuán
- Leandro Saraco

Código fuente: <https://github.com/Kajachuan/nlp3>

1. Objetivo del proyecto

El proyecto implementa un asistente conversacional orientado a consultas de compra de componentes electrónicos. El usuario puede pedir precio, disponibilidad conceptual o URL de compra de un componente; el sistema prioriza evidencia de un catálogo local y activa búsqueda web solo cuando la evidencia local no alcanza.

La solución apunta a una demostración de nivel preproducción: código modular, interfaz funcional, políticas básicas de seguridad, observabilidad, evaluación automatizada y artefactos de documentación para presentación.

2. Arquitectura general

La arquitectura combina una interfaz conversacional con un grafo de orquestación. El planner decide la ruta, la búsqueda local híbrida recupera candidatos desde catálogos JSON indexados en ChromaDB, un gate determinístico decide si corresponde fallback web y un verificador LLM produce la respuesta final con evidencia.

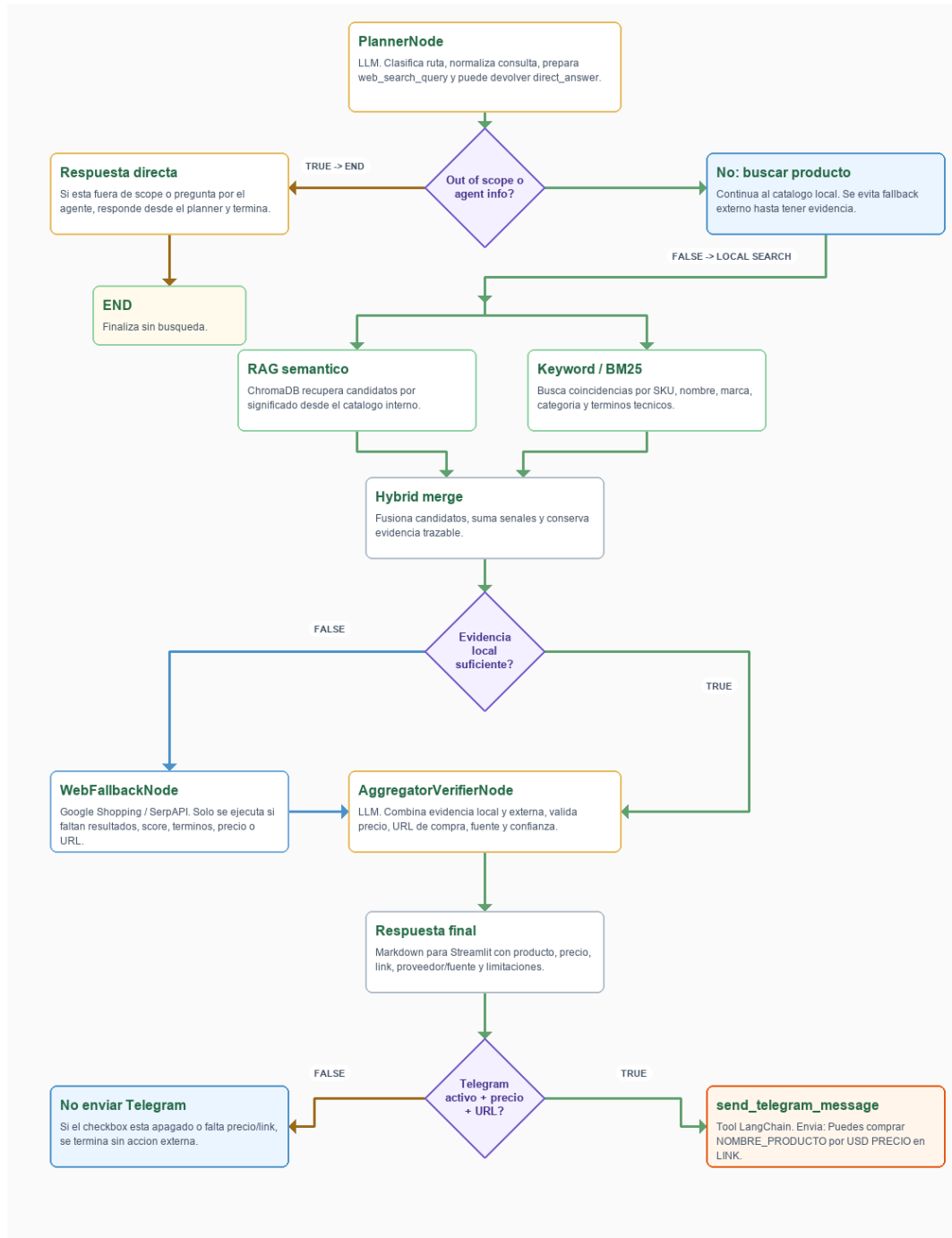


Figura 1. Diagrama de arquitectura..

Componente	Modulo	Responsabilidad
Streamlit UI	app/main.py	Chat, historial, controles de top_k/web_limit, método web, modelos por agente y Telegram opcional.
Orquestador	src/agents/orchestration/component_advisor.py	Grafo LangGraph con nodos planner, respuesta directa, búsqueda local, gate, fallback web, verificador y Telegram.
RAG local	src/rag/retrievers y src/vectorstore/chroma	ChromaDB persistente, retriever semantico, Keyword/BM25 y merge hibrido sobre 408 componentes.
Fallback web	src/tools/external/web_search_adapter.py	Adapter de web-search-tool para Tavily/Google Shopping/Google Search, con fallback offline cuando faltan credenciales.
Observabilidad	src/common/telemetry	Métricas por etapa, tokens, costo estimado, trazas Langfuse y auditoria JSONL.
Seguridad	src/policies/security.py	Validación de consultas vacías, largas, secretas, prompt injection básica y payloads peligrosos.

Flujo principal

1. El usuario ingresa una consulta en la UI de Streamlit.
2. El planner LLM clasifica la ruta: fuera de dominio, información del agente o búsqueda local.
3. Para consultas de compra, se ejecuta búsqueda local híbrida: RAG semántico en ChromaDB, Keyword/BM25 y merge ponderado.
4. El Evidence Gate valida si la evidencia local alcanza usando umbrales, cantidad de resultados, términos específicos y precio.
5. Si falta evidencia, se llama al fallback web con proveedores externos configurables.
6. El agregador/verificador LLM selecciona la mejor evidencia y arma la respuesta final.
7. Opcionalmente se envía una notificación por Telegram si hay producto, precio y URL.
8. Se registran métricas, trazas Langfuse y eventos de auditoría.

Se recomienda ver el archivo HTML del repositorio para una versión interactiva del flujo de datos:

<https://github.com/Kajachuan/nlp3/blob/main/llm-agent-project/docs/architecture.html>

3. Implementación técnica

Herramientas y librerías

Tecnología	Uso
Python + Streamlit	Interfaz de usuario conversacional y panel lateral de configuración.
LangGraph	Orquestación de nodos, rutas condicionales y estado compartido.
LangChain + Groq	Invocación de modelos LLM para planner, respuesta directa y verificador.
ChromaDB	Vectorstore persistente para búsqueda semántica local.
Keyword/BM25	Recuperación léxica complementaria para SKUs, nombres y términos técnicos.
web-search-tool	Busqueda externa por Tavily, Google Shopping o Google Search.
Langfuse	Trazabilidad, generaciones, spans, tokens, costo y errores.
Pytest + DeepEval	Evaluación reproducible y reportes HTML de métricas.

Modulos clave

- `factory.py` construye el pipeline y cachea el `ComponentAdvisorOrchestrator`.
- `component_advisor.py` define los contratos `PlannerDecision`, `AggregatorDecision` y `AdvisorResponse`, y compila el grafo `LangGraph`.
- `hybrid_retriever.py` combina score semántico y score lexicio con pesos 0.65/0.35 y bonus cuando un ítem aparece en ambas fuentes.
- `evidence_gate.py` evita usar web de forma innecesaria y fuerza fallback cuando faltan términos específicos o precio.
- `web_search_adapter.py` normaliza resultados externos y protege la demo con modo offline cuando faltan credenciales.
- `telegram_tool.py` queda encapsulado como tool y solo se ejecuta cuando la UI lo habilita y la respuesta final tiene evidencia de compra.

Datos e ingesta

El loader de catálogos lee todos los archivos JSON en `data/raw`, valida que cada ítem tenga SKU y rechaza duplicados. En la versión actual se indexan 408 componentes distribuidos en 9 archivos: sensores, resistores, capacitores, conectores, microcontroladores e integrados de distintos fabricantes.

La ingesta se puede ejecutar con `scripts/ingest_catalog_to_chroma.py`. La app también asegura la indexación al construir el pipeline si la colección persistente no está disponible.

Contratos de salida

Los prompts exigen JSON estructurado para planner y verificador. Esto permite medir rutas, registrar decisiones, controlar fallbacks y desacoplar la UI de los detalles internos del LLM.

Contrato	Campos principales
Planner	route, normalized_query, product_terms, web_search_query, needs_price, reason
Search result	product_name, sku, price, purchase_url, vendor, source, score, evidence
Aggregator	should_answer, product_name, price, purchase_url, confidence, answer, limitations, citations

4. Evaluación

La evaluación combina tests unitarios, pruebas de integración y dos reportes de métricas. El reporte manual con pytest es la referencia principal porque el README lo marca como última evaluación y sus umbrales pasan. El reporte DeepEval queda como evaluación complementaria y actualmente evidencia brechas en recall/contexto/robustez bajo esa metodología.

Metrica	Que mide	Score	Umbral	Estado
Recall@K	Si el producto esperado o equivalente aparece en el top-k local.	0.938	0.850	PASA
Context Recall	Si la respuesta final queda soportada por producto, precio, URL y fuente.	0.875	0.850	PASA
Abstencion correcta	Rechazo correcto de consultas fuera de dominio sin activar herramientas innecesarias.	1.000	0.900	PASA
Robustez	Caída de recall entre consultas limpias y consultas con paráfrasis/ruido.	0.917	0.900	PASA

Tabla 1. Resultados del reporte en un dataset de evaluación de 40 queries

Metrica complementaria	Score	Umbral	Estado
DeepEval Recall@K	0.781	0.850	FALLA
DeepEval Context Recall	0.797	0.850	FALLA
DeepEval Correct Abstention	1.000	0.900	PASA
DeepEval Robustness	0.708	0.900	FALLA

Tabla 2. Reporte complementario reports/deepeval_metrics_report.html.

La lectura conjunta muestra que el pipeline principal cumple los umbrales definidos para la entrega, mientras que DeepEval señala oportunidades de calibración del dataset, aceptación de SKUs equivalentes y recuperación ante consultas con ruido.

Cobertura de tests

- tests/unit cubre seguridad, runtime config, prompts, retrievers, Telegram, web search adapter y evidence gate.
- tests/e2e/test_real_api_flow.py valida el flujo contra API real cuando hay credenciales.
- tests/eval genera reportes HTML con dataset de 40 muestras.

5. Resultados y ejemplos

La app permite demostrar tres escenarios relevantes para la presentación: respuesta directa fuera de dominio, búsqueda local suficiente y búsqueda local con fallback web. En todos los casos se exponen métricas y resultados intermedios para explicar por qué el agente respondió de esa manera.

Escenario	Consulta ejemplo	Ruta esperada	Resultado
Producto local	Dame el precio del BME280	local_search -> local_hybrid_search -> aggregator_verifier	Responde con ítem del catálogo, precio en USD y link local de compra.
Necesidad tecnica	Necesito un capacitor de desacople de 100nF	local_search -> local_hybrid_search	Recupera capacitores cerámicos SMD y justifica la recomendación.
Producto no local	Precio de ACS71240	local_search -> web_fallback -> aggregator_verifier	Usa fallback web si el SKU específico no aparece en la evidencia local.
Fuera de dominio	Contame un chiste	out_of_domain -> direct_response	Rechaza la consulta sin RAG, keyword ni web.

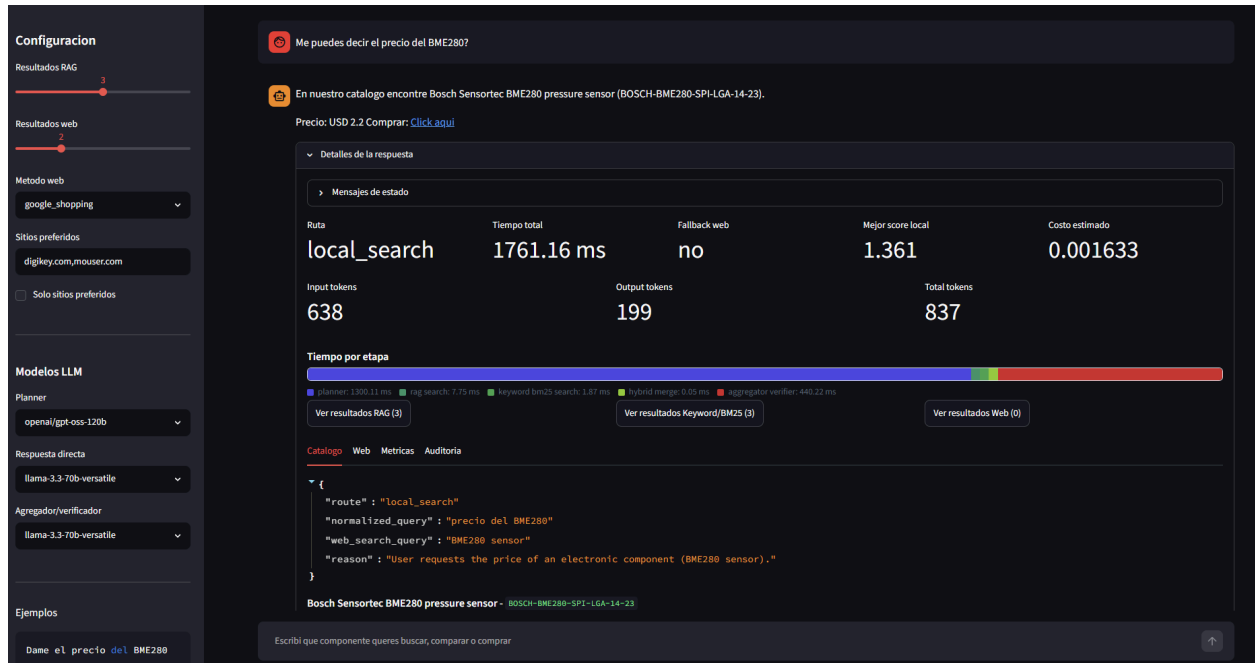


Figura 2. Interfaz Streamlit con chat, metricas y resultados intermedios.

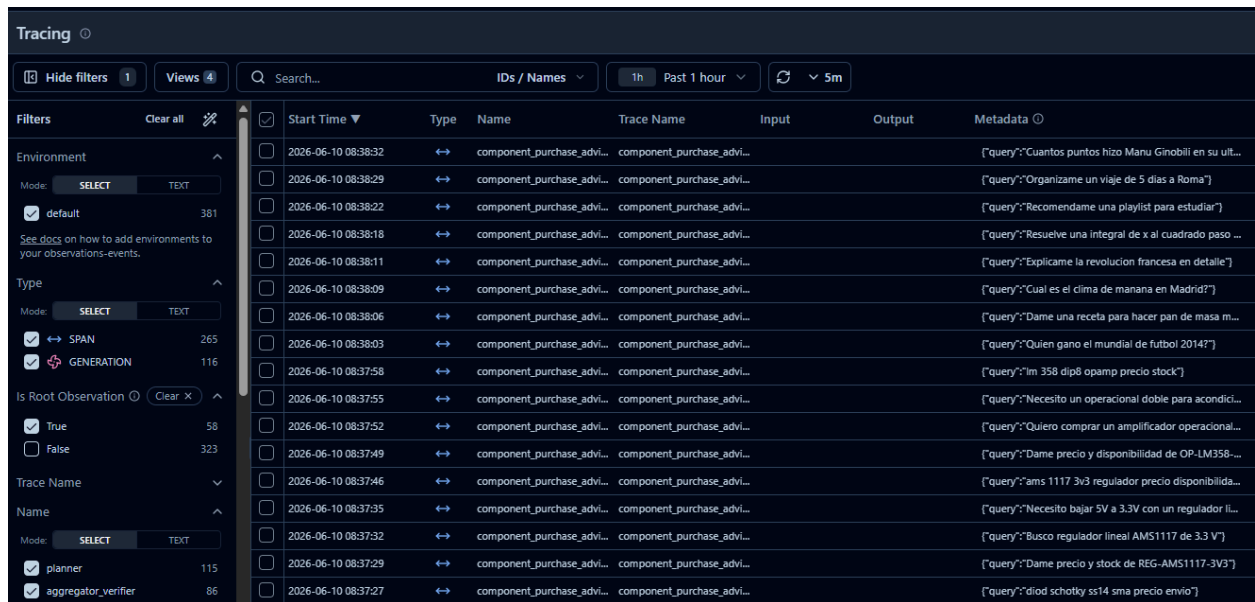


Figura 3. Dashboard de Langfuse usado para observar trazas y latencias.

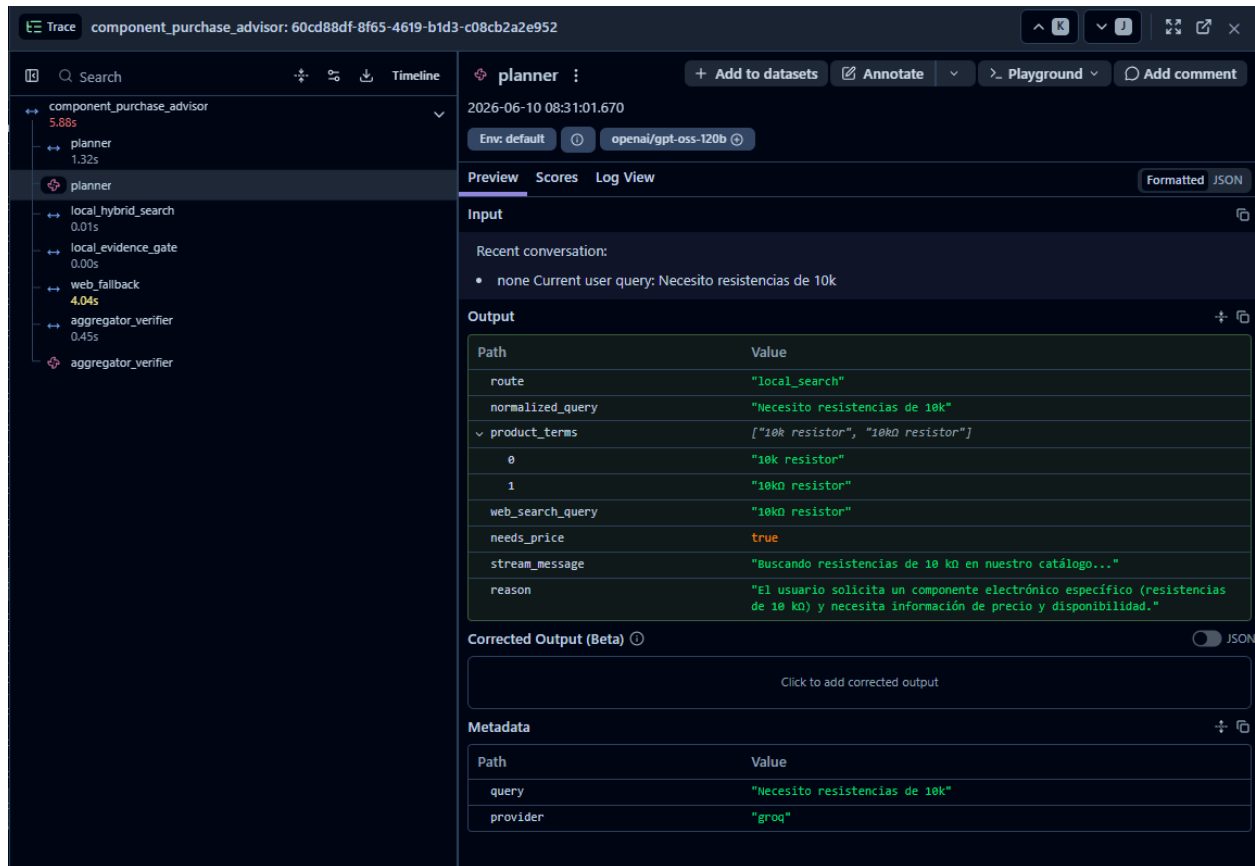


Figura 4. Observabilidad del flujo de datos en LangFuse.

6. Conclusiones y mejoras futuras

El repositorio implementa una POC completa y demostrable para la consigna: una aplicación conversacional con agentes coordinados, RAG, fallback web, observabilidad, seguridad básica y evaluación reproducible. La decisión de usar un gate determinístico antes del fallback web reduce costo y latencia, y facilita explicar el comportamiento del sistema durante la demo.

Mejoras futuras

- Recalibrar el dataset y los criterios de equivalencia de SKU para alinear la evaluación manual y DeepEval.
- Conectar el PretrainedIntentModel al grafo o reemplazarlo por un modelo preentrenado más pertinente si se exige inferencia adicional.
- Fortalecer políticas de salida: detección de PII, sanitización de enlaces y validación de proveedores.
- Agregar caché de resultados web y límites dinámicos de tokens por ruta.
- Expandir ejemplos de presentación con casos de error, latencia y comparación entre modelos.
- Automatizar generación de PDF final desde el DOCX para la entrega.

Anexo 1: Planificación del equipo

Previo a la realización del trabajo se planificó que tareas deben cubrir un equipo de profesionales de software para realizar el trabajo para alcanzar un nivel de producción.

Frente	Tareas	Responsable funcional
Orquestación y agentes	Planner, grafo LangGraph, verificador, contratos JSON y streaming de estado.	Backend
RAG y catalogo	Ingesta de JSON, ChromaDB, Keyword/BM25, merge híbrido y ajuste de ranking.	Ingeniero de IA/datos
Tools externas	Adapter web-search-tool, fallback offline y Telegram controlado por reglas.	Backend
Seguridad y auditoria	Validacion de input, limites, logs JSONL y metricas.	Backend / QA
Evaluacion	Dataset, pytest, DeepEval, reportes HTML y analisis de brechas.	QA
Documentación y demo	Arquitectura visual, informe, capturas, video demo y presentación.	Backend e Ingeniero de IA

Anexo 2. Comandos de ejecución

Accion	Comando
Instalar dependencias	<code>pip install -r requirements.txt</code>
Ingestar catalogo	<code>python scripts/ingest_catalog_to_chroma.py</code>
Ejecutar app	<code>streamlit run app/main.py</code>
Tests unitarios	<code>python -m pytest tests/unit</code>
Metricas	<code>python -m pytest tests/eval/test_metrics_evaluation.py -m eval_metrics --run-eval-metrics -s</code>

Para ejecutar las llamadas reales a LLM, Langfuse y búsqueda web se requiere configurar `.env` con `GROQ_API_KEY`, credenciales Langfuse y `SERPAPI_API_KEY` o el backend web elegido. Se recomienda ver `.env.example` dentro del repositorio.